Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3605731.3605886

RESEARCH-ARTICLE

Evaluation of OpenAI Codex for HPC Parallel Programming Models Kernel Generation

WILLIAM GODOY, Oak Ridge National Laboratory, Oak Ridge, TN, United States

PEDRO VALERO-LARA, Oak Ridge National Laboratory, Oak Ridge, TN, United States

KEITA TERANISHI, Oak Ridge National Laboratory, Oak Ridge, TN, United States

PRASANNA BALAPRAKASH, Oak Ridge National Laboratory, Oak Ridge, TN, United States

JEFFREY VETTER, Oak Ridge National Laboratory, Oak Ridge, TN, United States

Open Access Support provided by:

Oak Ridge National Laboratory

PDF Download
3605731.3605886.pdf
07 April 2026
Total Citations: 31
Total Downloads: 516

Published: 07 September 2023

Citation in BibTeX format

ICPP-W 2023: 52nd International Conference on Parallel Processing Workshops
August 7 - 10, 2023
UT, Salt Lake City, USA

ICPP Workshops '23: Proceedings of the 52nd International Conference on Parallel Processing Workshops (August 2023)
https://doi.org/10.1145/3605731.3605886
ISBN: 9798400708428

Evaluation of OpenAI Codex for HPC Parallel Programming Models Kernel Generation

William F. Godoy, Pedro Valero-Lara, Keita Teranishi, Prasanna Balaprakash, and Jeffrey S. Vetter
Oak Ridge National Laboratory
Oak Ridge, TN, USA
{godoywf},{valerolarap},{teranishik},{pbalapra},and{vetter}@ornl.gov

ABSTRACT

We evaluate AI-assisted generative capabilities on fundamental numerical kernels in high-performance computing (HPC), including AXPY, GEMV, GEMM, SpMV, Jacobi Stencil, and CG. We test the generated kernel codes for a variety of language-supported programming models, including (1) C++ (e.g., OpenMP [including offload], OpenACC, Kokkos, SyCL, CUDA, and HIP), (2) Fortran (e.g., OpenMP [including offload] and OpenACC), (3) Python (e.g., numpy, Numba, cuPy, and pyCUDA), and (4) Julia (e.g., Threads, CUDA.jl, AMDGPU.jl, and KernelAbstractions.jl). We use the GitHub Copilot capabilities powered by the GPT-based OpenAI Codex available in Visual Studio Code as of April 2023 to generate a vast amount of implementations given simple `<kernel> + <programming model> + <optional hints>` prompt variants. To quantify and compare the results, we propose a proficiency metric around the initial 10 suggestions given for each prompt. Results suggest that the OpenAI Codex outputs for C++ correlate with the adoption and maturity of programming models. For example, OpenMP and CUDA score really high, whereas HIP is still lacking. We found that prompts from either a targeted language such as Fortran or the more general-purpose Python can benefit from adding code keywords, while Julia prompts perform acceptably well for its mature programming models (e.g., Threads and CUDA.jl). We expect for these benchmarks to provide a point of reference for each programming model's community. Overall, understanding the convergence of large language models, AI, and HPC is crucial due to its rapidly evolving nature and how it is redefining human-computer interactions.

KEYWORDS

GPT, large language models, LLM, programming models, generative AI, numerical kernels, high-performance computing, HPC, GitHub Copilot,OpenAI Codex

ACM Reference Format:
William F. Godoy, Pedro Valero-Lara, Keita Teranishi, Prasanna Balaprakash, and Jeffrey S. Vetter. 2023. Evaluation of OpenAI Codex for HPC Parallel Programming Models Kernel Generation. In *52nd International Conference on Parallel Processing Workshops (ICPP-W 2023), August 07–10, 2023, Salt Lake City, UT, USA.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3605731.3605886

This manuscript has been authored by UT-Battelle LLC under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The publisher acknowledges the US government license to provide public access under the DOE Public Access Plan (https://energy.gov/downloads/doe-public-access-plan).

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of the United States government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.
ICPP-W 2023, August 07–10, 2023, Salt Lake City, UT, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0842-8/23/08…$15.00
https://doi.org/10.1145/3605731.3605886

1 INTRODUCTION

Since its initial release in 2020, the Generative Pre-trained Transformer 3 (GPT-3) [6] has signified a revolutionary step in the evolution of human-computer interactions. Developed by OpenAI,[1] GPT-3 is the third-generation prediction-based large language model (LLM) used for several AI-generated, human-like text applications. It has received praise for the high-quality results in several natural language processing (NLP) tasks [20] due in part to the unprecedented investment ($12 million USD) and size of its training model (175 billion parameters at 800 GB). Hence GPT-3 and its successor GPT-4[2] are defining several societal questions for the near future.

As we enter the exascale computing era, which is dominated by the extreme heterogeneity of hardware and programming models [46], AI-assisted code generation could play a key role in how we develop, deploy, and test software that targets high-performance computing (HPC) systems. Traditional human-readable code such as C++ [42], Fortran [2], Python [45], and more recently Julia [4], are a straightforward application for GPT-3 LLM capabilities that would help redefine software development. Hence, we need to understand the current state of practice, limitations, and potential of this new technology.

Our early experiences with OpenAI Codex, which is a GPT-3 descendant, came from using the GitHub Copilot[3] plugin available on Visual Studio Code[4] to generate relevant scientific kernels for HPC. Our goal is to assess and understand the impact of GPT-3's state-of-art capabilities in the overall interactive process for generating, optimizing, building, and testing well-established mathematical HPC kernels. For our assessment, we evaluate AXPY, general matrix-vector multiply (GEMV), general matrix-matrix multiply (GEMM), sparse matrix–vector multiplication (SpMV), 3D Jacobi stencil computations, and conjugate gradients (CG). We test the generation of these kernels over a variety of programming models that target CPU and GPU hardware. Our goal is to establish a benchmark and understand the status of *prompt* engineering in the OpenAI Codex when applied to these important kernels and programming models as the technology continues to evolve.

The rest of the paper is structured as follows: Section 2 provides an overview of related efforts that highlight the recent attention

[1] https://openai.com/
[2] https://openai.com/product/gpt-4
[3] https://github.com/features/copilot
[4] https://docs.github.com/en/copilot/getting-started-with-github-copilot?tool=vscode

to these topics in the broader area of computer science. We describe our prompt input pattern methodology for interacting with GitHub Copilot for the kernel generation process along with the proposed metric to score the quality of the suggested outputs in Section 3. Section 4 presents the results of our evaluation along with our findings on prompt trade-off options for each language, kernel, programming model, and additional keyword inputs on the generated outputs. We aim to understand the current status of the correctness, trade-offs, and overall value of LLMs for HPC practitioners who wish to adopt this technology. Conclusions and future directions are presented in Section 5. Appendix A provides the artifact description for the reproducibility of this study.

2 BACKGROUND

The availability of GPT-3 has led to increasing interest in its LLM capabilities and how they could be applied to a large variety of applications. Brown et al. [6] introduced the seminal paper on GPT-3's unprecedented results, which showed strong performance in NLP interactions such as translation, question-answering, and cloze tasks. They highlight the use and success rates of prompt-based [14, 15] zero, one, and few-shot learning (FSL) techniques when only a few data examples are available to train a model as opposed to other previously trained ML alternatives. Wang et al. [47] presented a comprehensive review of FSL and pointed out the "unreliable empirical risk minimizer" that, due to its nature, compensates for the lack of supervised information by using prior available knowledge. Floridi and Chiriatti [17] provide a commentary on the nature of GPT-3, its scope, and limits while outlining the social consequences of the "industrialization of automatic cheap production of good, semantic artifacts."

On the code generation side, Dehaerne et al. [10] provided a systematic review of applying ML methods in description-to-code, code-to-code, and code-to-description studies from the past 6 years. The review indicated limitations, including the variability in the quality of generated, mined source code and the need and availability of large datasets (e.g., GitHub sources). The review also highlighted that automatically generating data is a fast alternative for obtaining data but is only appropriate for certain contexts, such as *programming by example*. Recent work has also focused on using GitHub Copilot's *AI pair programmer*, which is based on OpenAI Codex and leverages the vast stores of source code hosted on GitHub for AI-assisted code generation. Chen et al. [8] provided an introduction and evaluation of Codex for its Python code-writing capabilities. They point out the current limitations and difficulty with "docstrings describing long chains of operations and with binding operations to variables." A major challenge noted is the overreliance on Codex by novice programmers. Importantly, Copilot uses a different Codex version than the one used in our study. Nguyen and Nadi [31] provided an empirical evaluation of GitHub Copilot suggestions for correctness and understandability to LeetCode questions in Java, JavaScript, Python, and C. Shortcomings included generating further simplified code that relies on undefined functions. Vaithilingam, Zhang, and Glassman [43] provided a human subject study that consisted of 24 participants on the usability of GitHub Copilot. They concluded that Copilot did not necessarily reduce the task completion time in common tasks, such as file editing, web scraping, and graph plotting, for experienced Python users. Sobania et al. [41] found little difference between Copilot and automatic generators that use genetic programming when applied to the PSB2 program synthesis benchmarks [19]. Imai [21] provided a preliminary assessment that showed that, although Copilot helps generate lines of code faster, the quality is lower when compared to human pair programming. Yetistiren et al. [49] assessed the correctness, validity, and efficiency of targeting the HumanEval problem dataset [8] with a high success rate.

From an educational perspective, Sarsa et al. [40] discussed Codex's effectiveness in generating programming exercises and explanations via Copilot, not without addressing the need for oversight in output quality. Finnie-Ansley et al. [16] reported that Codex ranks high when compared to a class of introductory computer science students taking programming tests. Similarly, Denny et al. [11] discussed the pedagogical value of Copilot for testing their answers and for its prompt nature in introductory programming questions. Wermelinger [48] discussed important questions about how teaching programming will evolve. Similarly, Brennan and Lesage [5] discussed the educational opportunities for undergraduate engineering and the need for students to have a strong intuition for software development when using AI-assisted tools.

Pierce et al. [37] assessed security aspects of Copilot's code contributions in a large sample, and they found relatively high vulnerability rates. They also concluded that there is a need to quantify the limits of generated code in low-level scenarios, and Copilot must be paired with security-aware tooling.

Overall, there is a common thread in the relevant literature: the GPT-based tools are here to stay and will impact almost every aspect of human-computer interaction. However, we are at an inflection point where more exploratory research is needed when assessing AI code-generating capabilities, and their technical, economic, and social implications must be carefully explored and better understood. To the best of our knowledge there is no reported work on the application of current Copilot capabilities for HPC kernels.

3 METHODOLOGY

We evaluate prompt outputs for parallel programming models available in four different languages: C++, Fortran, Python, and Julia. Our methodology has two aspects: (1) the Copilot code suggestion generation from prompt queries based on kernels and the parallel programming model for each language and (2) a simple metric to evaluate the correctness of the results. As stated in their documentation,[5] GitHub Copilot is "trained on all languages that appear in public repositories. For each language, the quality of suggestions you receive may depend on the volume and diversity of training data for that language." Hence, we investigate whether or not these results correlate with the expected availability of correct programming models and public open-source code examples [30] from metrics such as: (1) the number of repositories per language on GitHub (GitHut[6]) and (2) the TIOBE[7] index. Hence, C++ and Python codes are expected to have wider availability than Julia and Fortran. Nevertheless, because of the general-purpose nature of

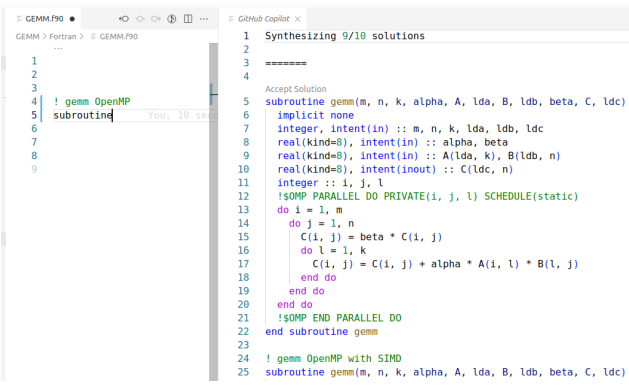
[5] https://docs.github.com/en/copilot/overview-of-github-copilot/about-github-copilot-for-individuals
[6] https://madnight.github.io/githut/#/pull_requests/2023/1
[7] https://www.tiobe.com/tiobe-index/

C++ and Python, this metric might not be as relevant as it is for scientific and mathematical applications written in Fortran or Julia.

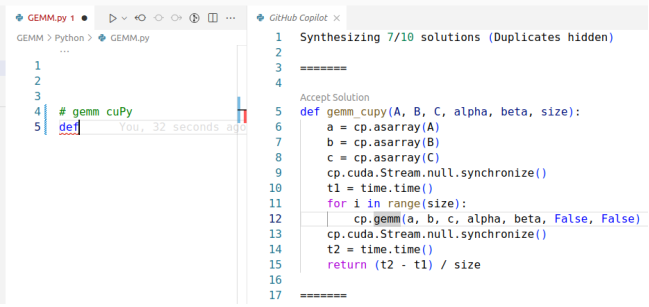
We assess the outputs from each prompt and categorize based on their level of correctness. The rest of the section describes the experiment setup and the proposed metric to quantify the quality of the AI-generated results.

3.1 Experiment setup

We select a simple prompt base structure for each targeted language. Visual Studio Code recognizes the targeted language through the appropriate support plugin and the currently opened file based on the language extension (e.g., cpp, f90, py, jl) and makes it part of the prompt prefix. The rest of the prompt is generated through a comment line complemented with optional code. Pressing `<Control> + <Enter>` results in a separate window that contains a maximum of 10 code suggestions. Given the highly sensitive nature of the LLM, even one additional keyword could heavily influence the output.



(a)



(b)

Figure 1: GitHub Copilot prompt interface with optional language keywords for (a) Fortran's `subroutine` and (b) Python's `def`.

Therefore, our simple prompt queries can be described by using the following structure:

- <kernel> <programming-model>
- <kernel> <programming-model> <optional keywords: `function`, `subroutine`, and `def`>

Table 1 shows the programming language and model combinations used for this study. The additional post-fix optional keywords correspond to our attempt to add more information to the prompt query to obtain better-quality suggestions. As shown in Section 4, C++ model sensitivity can vary by using the word *function* (not a language keyword), whereas Fortran and Python are consistently sensitive to the `subroutine` and `def` keywords. We also found little sensitivity in Julia prompts when adding a post fix (e.g., `function`), hence we did not add it in this study.

Language	Programming model	Post fix
C++	OpenMP [36]	offload, function
	OpenACC [35]	function
	Kokkos [7]	function
	CUDA [33]	function
	HIP [1]	function
	Thrust [34]	function
Fortran	OpenMP	offload, subroutine
	OpenACC	subroutine
Python	numpy [44]	def
	Numba [29]	def
	pyCUDA [26]	def
	cuPy [32]	def
Julia	Threads [27]	
	CUDA [3]	
	AMDGPU [39]	
	KernelAbstractions [9]	

Table 1: Scope of our experimental setup applied for each kernel in terms of language and targeted parallel programming model.

3.2 Correctness metric

To evaluate the correctness of the generated suggestions, we propose a simple approach that is based on our observations more than on any particular formalism. We consider five levels of correctness and proficiency labels between [0], or *non-knowledge*, and [1], or *expert*, when observing the suggested answers given by Copilot as those illustrated in Figures 1a and 1b for each combination in Table 1.

- 0 *non-knowledge*: No code at all or not a single correct code.
- 0.25 *novice*: One correct code, but it includes other several correct or incorrect programming models (e.g., OpenACC suggestions in an OpenMP prompt).
- 0.5 *learner*: One correct code, and there are other incorrect codes, but all of them are using the requested programming model.
- 0.75 *proficient*: All codes are correct and using the programming model requested.
- 1 *expert*: Only one piece of code is provided, and it is totally correct.

4 RESULTS

We present our analysis for each selected language and kernel and discuss some observations from our initial interaction with GitHub Copilot. As shown in Figures 1a and 1b, for each combination listed in Table1, we obtain a set of suggestions for each prompt. This approach enables us to evaluate the effectiveness of OpenAI Codex

in generating accurate and proficient code. Notably, none of the experiments reached a perfect score of 1, which illustrates the stochastic status of the current LLM when applied to a particular area.

4.1 C++

Table 2 shows the results for all of our C++ experiments, and Figure 2 illustrates the results according to the different kernels and programming models.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi	CG
Prefix <kernel>						
OpenMP	0.75	0.5	0.5	0.5	0	0.25
OpenMP offload	0.5	0.5	0.5	0.25	0.25	0
OpenACC	0.5	0	0.25	0	0	0
Kokkos	0.5	0	0	0	0.25	0
CUDA	0.75	0.75	0.75	0	0	0.25
HIP	0.75	0	0	0	0.25	0
Thrust	0.25	0	0	0	0	0
SyCL	0.75	0.25	0	0	0	0
Prefix <kernel> Postfix `function`						
OpenMP	0.75	0.75	0.75	0.25	0.25	0.25
OpenMP offload	0.5	0.5	0.5	0.25	0.25	0
OpenACC	0.5	0.5	0.5	0.25	0	0
Kokkos	0.75	0.25	0.25	0	0.25	0
CUDA	0.75	0.25	0	0	0	0
HIP	0.75	0	0	0	0.25	0
Thrust	0.5	0	0.25	0	0	0
SyCL	0.75	0.5	0.25	0	0	0

Table 2: Metric assessment for GitHub Copilot's C++ outputs using the input prompt pattern `<kernel> <programming model> (function)`.

As shown, the best results are achieved for the AXPY kernel. We see a clear trend in these results (Figure 2 [left]): the more complex the kernel, the fewer quality results are obtained. Although we see a level between *learner* and *proficient* for the AXPY kernel (the simplest kernel), for CG (the most complicated kernel), the level is close to *non-knowledge*. For the programming models, we see better results for OpenMP and CUDA. This could be due to the maturity of these programming models compared to others and their availability in public code. We note that the use of `function` as part of the prompt is very beneficial and increments the quality of the results in the dense matrix cases. Sparse matrices and high-level algorithms (e.g., SpMV, Jacobi, and CG) do not show much improvement. However, a different trend is observed for CUDA, for which, instead of improving the level of proficiency, it actually decreases the quality of the answer. This is perhaps because the word `function` is not used for CUDA codes. In our experimentation, which is not shown here, using the words "kernel" or "`__global__`" led to better code generation quality. This is an example of how important it is to adapt the language used for the prompts to the particularities or syntax habitually used by such a community.

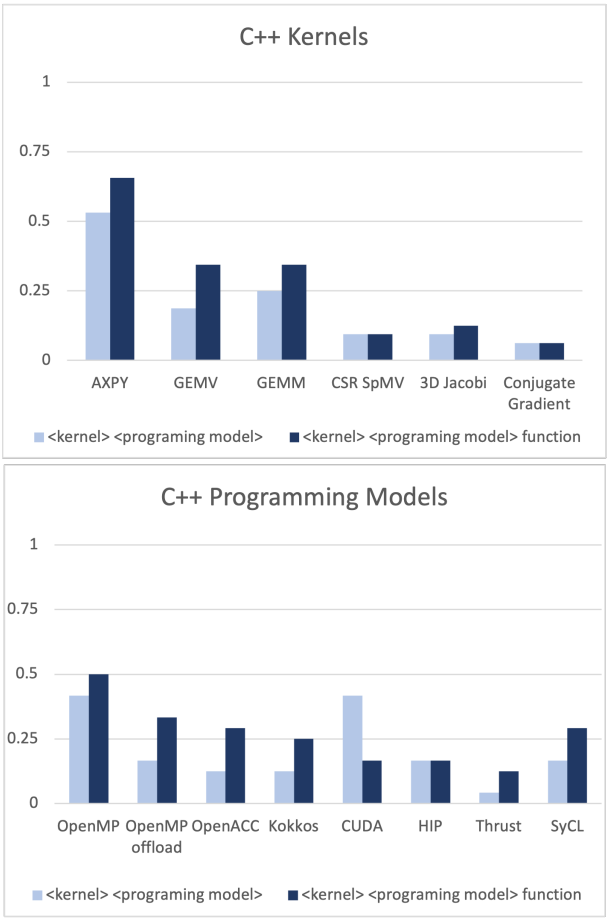


Figure 2: Results for C++ kernels (top) and programming models (bottom).

Results also show that high-level programming model prompts from Kokkos, Thrust, and SyCL perform poorly over several kernels. We view these results as a reflection of the user community size of these high-level abstractions. Over several instances, we observed that many wrong answers or no answers at all dominate as the kernel becomes more complex. Notably, large benchmark repositories are available (e.g., HecBench [22, 23]), from which some of the responses originate.

4.2 Fortran

Fortran is of particular interest in this analysis owing to its importance in HPC and scientific computing. Despite not being a mainstream language in terms of code availability, Copilot can provide some good results because of Fortran's domain-specific nature and legacy.

As shown in Table 3, using an "optimized" prompt and the `subroutine` keyword is particularly beneficial in this case. Not using it leads to very poor results, with the AXPY OpenMP case being the only exception due to its simplicity and availability. We observe a trend similar to the one we saw in the C++ case: the more

mature solutions such as OpenMP and OpenACC provide better results for parallel codes that use Fortran.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi	CG
Prefix <kernel>						
OpenMP	0.75	0	0	0	0	0
OpenMP offload	0	0	0	0	0	0
OpenACC	0	0	0	0	0	0
Prefix <kernel> Postfix `subroutine`						
OpenMP	0.75	0.25	0.25	0.5	0.5	0.25
OpenMP offload	0.25	0.25	0.25	0.25	0.5	0.25
OpenACC	0.25	0.25	0.25	0.25	0.25	0.25

Table 3: Metric assessment for Copilot's outputs using the input prompt pattern `<kernel> <programming model> (subroutine)` for Fortran.

Figure 3 illustrates the uniform characteristics in the responses across kernels. Fortran is used in many legacy HPC applications, and with so many available Fortran codes to draw from, it is not surprising that Copilot can obtain correct kernel implementations with higher complexity kernels (e.g., CG, Jacobi) in addition to the simpler ones (e.g., AXPY, GEMM).

4.3 Python

Python is one of the most-used, general-purpose languages in industry, research, and AI and also has an important role in education and as a major target of AI-generative code, as documented in Section 2. Although neither a parallel programming model nor part of the standard, we consider `numpy` in our evaluation because it is the de-facto standard for scientific computing in Python.

Table 4 shows the results from our evaluation. Similar to Fortran, we observe that the quality improves dramatically when adding the Python `def` keyword to clarify that we are looking for functions in the language. Overall, Copilot can generate acceptable numpy, cuPy, and pyCUDA implementations across several kernels, but Numba falls behind. Perhaps this is an indication that cuPy and pyCUDA are more popular alternatives in the community via lightweight layers on top of compiled C or CUDA code, rather than using Numba's just-in-time approach on top of LLVM. Notably, Numba recently deprecated support for AMD GPU hardware, hence reinforcing the idea that GPU kernels written in Python favor CUDA-like implementations. Interestingly, successful GPU instances (e.g., pyCUDA and cuPy) include a correct raw CUDA kernel source code as a user-defined kernel. These instances are described in pyCUDA [25] and cuPy [38] documentation, respectively.

As seen in Figure 4, the resulting kernels can have varying degrees of success, but most return at least one correct answer. This is perhaps attributed to the wide availability of Python and numpy code in public repositories. The trend also confirms the lack of correct Numba results, and perhaps it is an opportunity to exploit its pure-Python nature. Although, as recently highlighted by Kailasa et al. [24], writing Numba code for complex algorithms can be as challenging as using a compiled language.

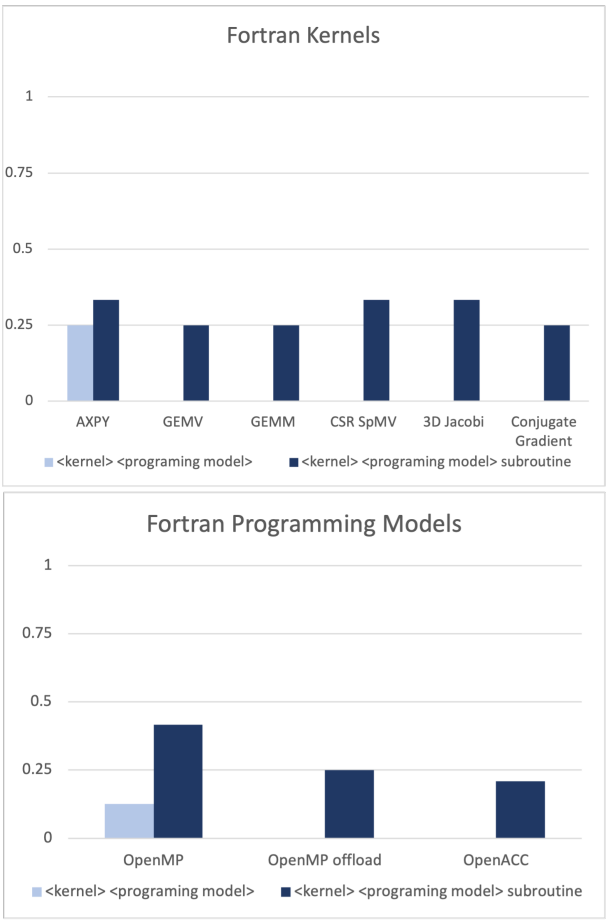


Figure 3: Results for Fortran kernels (top) and programming models (bottom).

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi	CG
Prefix <kernel>						
numpy	0.25	0	0	0	0	0
CuPy	0	0	0.25	0	0	0
pyCUDA	0	0	0	0	0	0
Numba	0	0	0	0	0	0
Prefix <kernel> Postfix `def`						
numpy	0.75	0.25	0.25	0.5	0.5	0.75
CuPy	0.5	0.25	0.25	0.25	0.25	0.25
pyCUDA	0.5	0.25	0.5	0.5	0.25	0
Numba	0.25	0	0	0	0	0

Table 4: Metric assessment for Copilot's outputs using the input prompt pattern `<kernel> <programming model> (def)` for Python.

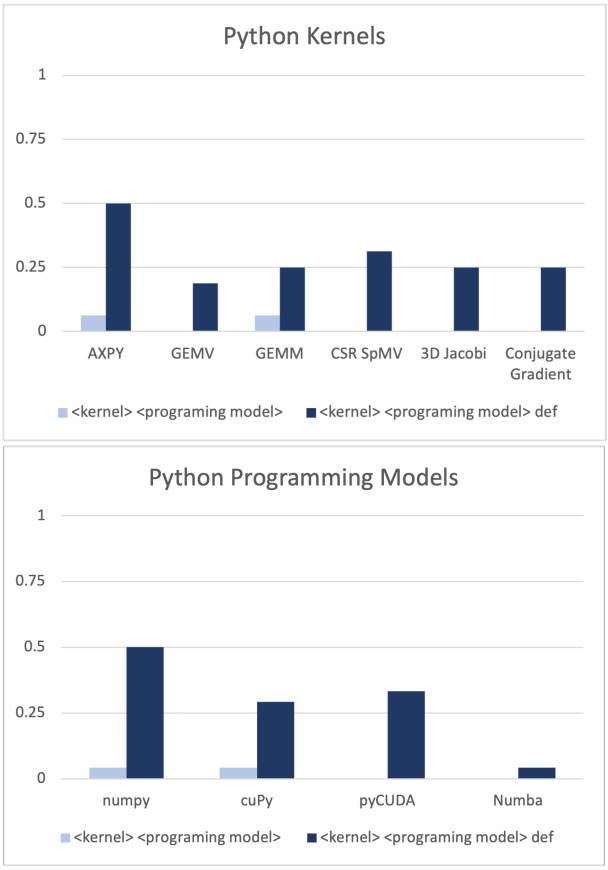


Figure 4: Results for Python kernels (top) and programming models (bottom).

4.4 Julia

We included the Julia language in our experiments because of its mathematical and performance focus. Julia provides an interesting proposition for building a dynamic language on top of LLVM and is heavily influenced by Fortran in its syntax for targeting scientific computing problems. Julia provides an accessible programming model for CPU Threads, which is part of the base language, for vendor-specific CUDA.jl and AMDGPU.jl, and for KernelAbstraction.jl for portable kernels across vendors. In previous work [18], we showed promising results, albeit with some gaps, in a simple performance comparison on CPU and GPU runs.

Table 5 and Figure 5 show the results for each kernel and programming model in our evaluation. For each programming model, we see a proficiency level between *novice* and *learner* for Threads (part of the base language) and CUDA.jl, which are the most used and mature programming models. AMDGPU.jl and KernelAbstractions.jl rank lower, which correlates with their novelty and the more limited availability of example code that targets non-NVIDIA GPU hardware.

Like with C++, a more complex kernel led to fewer correct results. For example, we could not get an appropriate implementation for the CG kernel (a multikernel algorithm). Unsurprisingly, as in the case of Fortran, Julia's mathematical nature allowed the OpenAI Codex to find and suggest appropriate solutions in other kernels, despite Julia being a relatively new language with fewer publicly available codes, especially compared to C++ and Python. Notably, our results did not necessarily improve by adding more information to the prompt (e.g., the `function` language keyword). This could be an advantage for domain-specific syntax, as in the case of Fortran, because most existing codes have a very targeted use with fewer words.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi	CG
Prefix <kernel>						
Threads	0.75	0.25	0.5	0	0	0
CUDA	0.75	0.5	0.5	0.25	0.25	0
AMDGPU	0	0	0	0.25	0	0
KernelAbstractions	0.25	0.25	0.25	0.25	0.25	0

Table 5: Metric assessment for Copilot's outputs using the input prompt pattern `<kernel> <programming model>` for Julia.

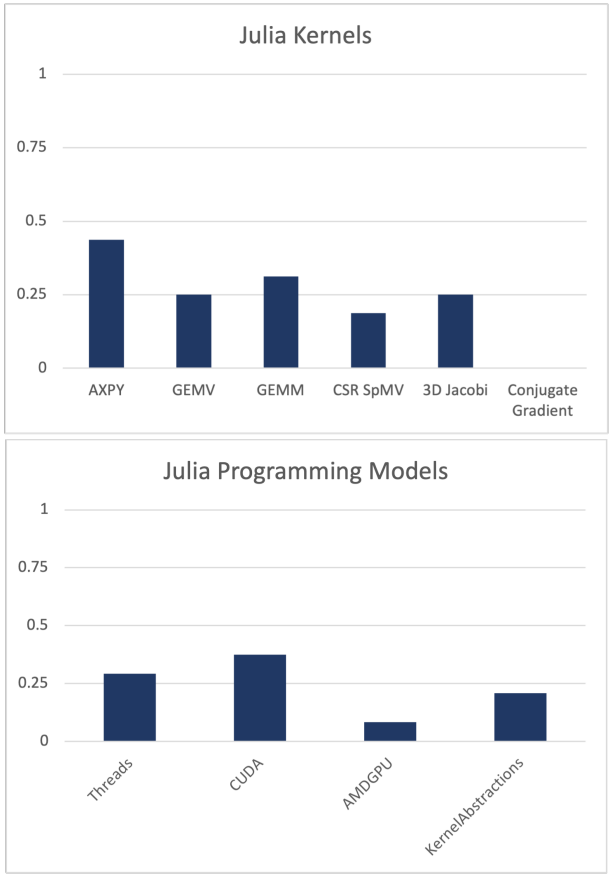


Figure 5: Results for Julia kernels (top) and programming models (bottom).

4.5 Discussion

To gain insight into the performance of OpenAI Codex across different languages and kernels, we collated the results in Figure 6. As depicted in the graph, the complexity of the kernel directly impacts the quality of the results obtained. In other words, it becomes increasingly challenging to achieve acceptable results as the kernel's complexity increases.

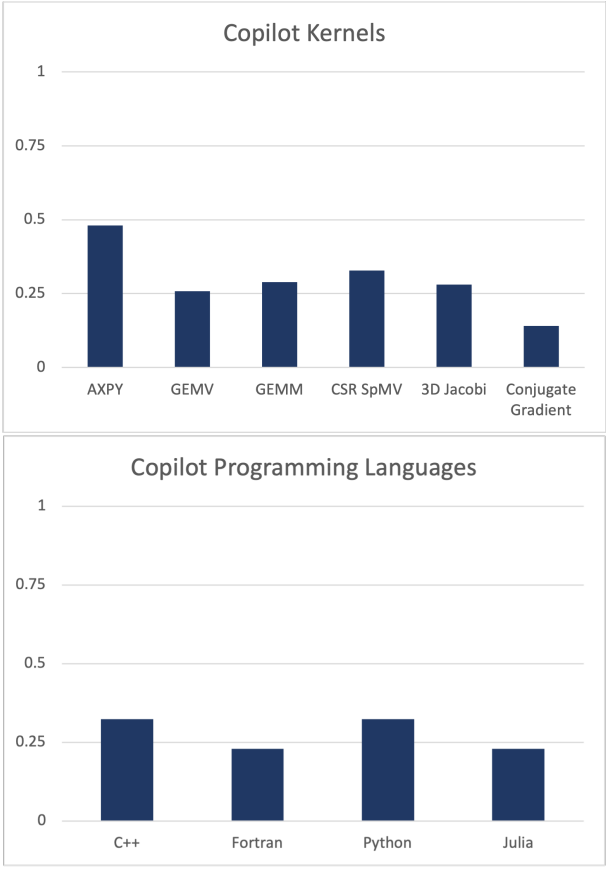


Figure 6: Overall results for Copilot kernels (top) and programming languages (bottom).

We see a relatively low level of proficiency in OpenAI Codex with an average of *novice* level for the languages and kernels tested. We see a slightly higher quality for the more popular C++ and Python languages than for Julia or Fortran. This may be related to the popularity, maturity, and accessibility of public codes implemented in those languages. However, Fortran and Julia provide very acceptable results. The results confirm that the question is more about targeted quality than quantity in a specific domain.

We summarize our results as follows:

- As code complexity increases, obtaining acceptable results becomes more challenging.
- Generating high-quality multistep or multikernel codes (e.g., CG) can be difficult.
- Using keywords can improve the proficiency of the answers, but it is essential to choose the correct words that are specific and sensitive to each programming language/model or community.
- Although the popularity or accessibility of a programming language or public code can be important, less popular languages can also provide good results owing to their targeted nature.

5 CONCLUSIONS AND FUTURE DIRECTIONS

We carried out an initial study to evaluate the current capacity of OpenAI Codex via Copilot for the generation of HPC numerical kernels that target parallel programming models in C++, Fortran, Python, and Julia. Despite current limitations, we believe that generative AI can have an extraordinary and beneficial impact on HPC software development, maintenance, and education in the future.

The research community still has several gaps to understand, and one such gap is the need for a more comprehensive taxonomy, akin to natural language, to evaluate accuracy and trustworthiness. Although our proposed taxonomy was beneficial for this initial study, it is imperative that such metrics be expanded to create a widely accepted methodology that the entire community can use. Therefore, a standard and recognized approach is needed to ensure uniformity when evaluating results across studies.

The emergence of LLM technologies such as GPT-3 and other generative AI tools presents significant questions about their integration into the future HPC software development ecosystem. For instance, can a human-in-the-loop and compiler be incorporated to refine initial LLM suggestions? Can metadata-rich suggestions be incorporated to facilitate a human decision-making process? Additionally, how do significant HPC software modernization initiatives similar to DARPA's High Productivity Computing Systems [12] or the US Department of Energy's Exascale Computing Project [13, 28] incorporate these novel tools?

The automation of ecosystem features, such as building systems, packaging, validation & verification, reproducibility, and continuous integration/continuous deployment pipelines, could significantly impact the HPC community. The technologies that put today's imperfect information closer to the human in question could redefine the educational aspects of HPC. Therefore, it is crucial to understand how each community can leverage these revolutionary capabilities to further advance their respective domains.

ACKNOWLEDGMENTS

This work is funded by Bluestone, an X-Stack project in the DOE Advanced Scientific Computing Office with program manager Hal Finkel.

REFERENCES

[1] AMD. 2022. AMD ROCm v5.2 Release. https://rocmdocs.amd.com/en/latest/Current_Release_Notes/Current-Release-Notes.html#amd-rocm-v5-2-release
[2] J. W. Backus and W. P. Heising. 1964. Fortran. *IEEE Transactions on Electronic Computers* EC-13, 4 (1964), 382–385. https://doi.org/10.1109/PGEC.1964.263818
[3] Tim Besard, Christophe Foket, and Bjorn De Sutter. 2018. Effective Extensible Programming: Unleashing Julia on GPUs. *IEEE Transactions on Parallel and Distributed Systems* (2018). https://doi.org/10.1109/TPDS.2018.2872064 arXiv:1712.03112 [cs.PL]

[4] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (Jan. 2017), 65–98. https://doi.org/10.1137/141000671 arXiv:http://dx.doi.org/10.1137/141000671

[5] Robert W. Brennan and Jonathan Lesage. 2023. Exploring the Implications of OpenAI Codex on Education for Industry 4.0. In *Service Oriented, Holonic and Multi-Agent Manufacturing Systems for Industry of the Future*, Theodor Borangiu, Damien Trentesaux, and Paulo Leitão (Eds.). Springer International Publishing, Cham, 254–266.

[6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[7] H. Carter Edwards, Christian R. Trott, and Daniel Sunderland. 2014. Kokkos: Enabling manycore performance portability through polymorphic memory access patterns. *J. Parallel and Distrib. Comput.* 74, 12 (2014), 3202–3216. https://doi.org/10.1016/j.jpdc.2014.07.003 Domain-Specific Languages and High-Level Frameworks for High-Performance Computing.

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[9] Valentin Churavy, Dilum Aluthge, Lucas C Wilcox, James Schloss, Simon Byrne, Maciej Waruszewski, Julian Samaroo, Ali Ramadhan, Meredith, Simeon Schaub, Jake Bolewski, Anton Smirnov, Charles Kawczynski, Chris Hill, Jinguo Liu, Oliver Schulz, Oscar, Páll Haraldsson, Takafumi Arakaki, and Tim Besard. 2022. *JuliaGPU/KernelAbstractions.jl: v0.8.3*. https://doi.org/10.5281/zenodo.6742177

[10] Enrique Dehaerne, Bappaditya Dey, Sandip Halder, Stefan De Gendt, and Wannes Meert. 2022. Code Generation Using Machine Learning: A Systematic Review. *IEEE Access* 10 (2022), 82434–82455. https://doi.org/10.1109/ACCESS.2022.3196347

[11] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (Toronto ON, Canada) *(SIGCSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1136–1142. https://doi.org/10.1145/3545945.3569823

[12] Jack Dongarra, Robert Graybill, William Harrod, Robert Lucas, Ewing Lusk, Piotr Luszczek, Janice Mcmahon, Allan Snavely, Jeffrey Vetter, Katherine Yelick, Sadaf Alam, Roy Campbell, Laura Carrington, Tzu-Yi Chen, Omid Khalili, Jeremy Meredith, and Mustafa Tikir. 2008. DARPA's HPCS Program: History, Models, Tools, Languages. In *Advances in COMPUTERS*. Advances in Computers, Vol. 72. Elsevier, 1–100. https://doi.org/10.1016/S0065-2458(08)00001-6

[13] Jack Dongarra et al. 2011. The International Exascale Software Project roadmap. *The International Journal of High Performance Computing Applications* 25, 1 (2011), 3–60. https://doi.org/10.1177/1094342010391989 arXiv:https://doi.org/10.1177/1094342010391989

[14] Li Fei-Fei, R. Fergus, and P. Perona. 2006. One-shot learning of object categories. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 28, 4 (2006), 594–611. https://doi.org/10.1109/TPAMI.2006.79

[15] Michael Fink. 2004. Object Classification from a Single Example Utilizing Class Relevance Metrics. In *Advances in Neural Information Processing Systems*, L. Saul, Y. Weiss, and L. Bottou (Eds.), Vol. 17. MIT Press. https://proceedings.neurips.cc/paper_files/paper/2004/file/ef1e491a766ce3127556063d49bc2f98-Paper.pdf

[16] James Finnie-Ansley, Paul Denny, Brett A. Becker, Andrew Luxton-Reilly, and James Prather. 2022. The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. In *Proceedings of the 24th Australasian Computing Education Conference* (Virtual Event, Australia) *(ACE '22)*. Association for Computing Machinery, New York, NY, USA, 10–19. https://doi.org/10.1145/3511861.3511863

[17] Luciano Floridi and Massimo Chiriatti. 2020. GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines* 30 (2020), 681–694.

[18] William F. Godoy, Pedro Valero-Lara, T. Elise Dettling, Christian Trefftz, Ian Jorquera, Thomas Sheehy, Ross G. Miller, Marc Gonzalez-Tallada, Jeffrey S. Vetter, and Valentin Churavy. 2023. Evaluating performance and portability of high-level programming models: Julia, Python/Numba, and Kokkos on exascale nodes. arXiv:2303.06195 [cs.DC]

[19] Thomas Helmuth and Peter Kelly. 2021. PSB2: The Second Program Synthesis Benchmark Suite. In *Proceedings of the Genetic and Evolutionary Computation Conference* (Lille, France) *(GECCO '21)*. Association for Computing Machinery, New York, NY, USA, 785–794. https://doi.org/10.1145/3449639.3459285

[20] Julia Hirschberg and Christopher D. Manning. 2015. Advances in natural language processing. *Science* 349, 6245 (2015), 261–266. https://doi.org/10.1126/science.aaa8685 arXiv:https://www.science.org/doi/pdf/10.1126/science.aaa8685

[21] Saki Imai. 2022. Is GitHub Copilot a Substitute for Human Pair-Programming? An Empirical Study. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 319–321. https://doi.org/10.1145/3510454.3522684

[22] Zheming Jin. 2021. *The Rodinia Benchmarks in SYCL*. Technical Report. Oak Ridge National Lab.(ORNL), Oak Ridge, TN (United States).

[23] Zhemin Jin. 2023. Hecbench. https://github.com/zjin-lcf/HeCBench.

[24] Srinath Kailasa, Tingyu Wang, Lorena A. Barba, and Timo Betcke. 2023. PyExaFMM: an exercise in designing high-performance software with Python and Numba. arXiv:2303.08394 [cs.SE]

[25] Andreas Klöckner. [n. d.]. pycuda 2022.2.2 documentation. https://documen.tician.de/pycuda/. Accessed: 2023-04-20.

[26] Andreas Klöckner, Nicolas Pinto, Yunsup Lee, Bryan Catanzaro, Paul Ivanov, and Ahmed Fasih. 2012. PyCUDA and PyOpenCL: A scripting-based approach to GPU run-time code generation. *Parallel Comput.* 38, 3 (2012), 157–174. https://doi.org/10.1016/j.parco.2011.09.001

[27] Tobias Knopp. 2014. Experimental multi-threading support for the Julia programming language. In *2014 First Workshop for High Performance Technical Computing in Dynamic Languages*. IEEE, 1–5.

[28] Douglas Kothe, Stephen Lee, and Irene Qualters. 2019. Exascale Computing in the United States. *Computing in Science & Engineering* 21, 1 (2019), 17–29. https://doi.org/10.1109/MCSE.2018.2875366

[29] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. 2015. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*. 1–6.

[30] Dongdong Lu, Jie Wu, Yongxiang Sheng, Peng Liu, and Mengmeng Yang. 2020. Analysis of the popularity of programming languages in open source software communities. In *2020 International Conference on Big Data and Social Sciences (ICBDSS)*. 111–114. https://doi.org/10.1109/ICBDSS51270.2020.00033

[31] Nhan Nguyen and Sarah Nadi. 2022. An Empirical Evaluation of GitHub Copilot's Code Suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories* (Pittsburgh, Pennsylvania) *(MSR '22)*. Association for Computing Machinery, New York, NY, USA, 1–5. https://doi.org/10.1145/3524842.3528470

[32] ROYUD Nishino and Shohei Hido Crissman Loomis. 2017. Cupy: A numpy-compatible library for nvidia gpu calculations. *31st confernce on neural information processing systems* 151, 7 (2017).

[33] NVIDIA. 2022. CUDA Toolkit Documentation - v11.7.0. https://developer.nvidia.com/cuda-toolkit

[34] NVIDIA. 2022. The API reference guide for Thrust, the CUDA C++ template library. https://docs.nvidia.com/cuda/thrust/index.html

[35] OpenACC Architecture Review Board. 2020. OpenACC Application Program Interface Version 3.1. https://www.openacc.org/sites/default/files/inline-images/Specification/OpenACC-3.1-final.pdf

[36] OpenMP Architecture Review Board. 2021. OpenMP Application Program Interface Version 5.2. https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf

[37] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. 754–768. https://doi.org/10.1109/SP46214.2022.9833571

[38] Inc. Preferred Networks and Inc. Preferred Infrastructure. [n. d.]. CuPy – NumPy & SciPy for GPU. https://docs.cupy.dev/en/stable/. Accessed: 2023-04-20.

[39] Julian Samaroo, Valentin Churavy, Wiktor Phillips, Ali Ramadhan, Jason Barmparesos, Julia TagBot, Ludovic Räss, Michel Schanen, Tim Besard, Anton Smirnov, Takafumi Arakaki, Stephan Antholzer, Alessandro, Chris Elrod, Matin Raayai, and Tom Hu. 2022. *JuliaGPU/AMDGPU.jl: v0.4.1*. https://doi.org/10.5281/zenodo.6949520

[40] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models *(ICER '22)*. Association for Computing Machinery, New York, NY, USA, 27–43. https://doi.org/10.1145/3501385.3543957

[41] Dominik Sobania, Martin Briesch, and Franz Rothlauf. 2022. Choose Your Programming Copilot: A Comparison of the Program Synthesis Performance of Github Copilot and Genetic Programming. In *Proceedings of the Genetic and Evolutionary Computation Conference* (Boston, Massachusetts) *(GECCO '22)*. Association for Computing Machinery, New York, NY, USA, 1019–1027. https://doi.org/10.1145/3512290.3528700

[42] Bjarne Stroustrup. 2013. *The C++ programming language*. Pearson Education.

[43] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) *(CHI EA '22)*. Association for Computing Machinery, New York, NY, USA, Article 332, 7 pages. https://doi.org/10.1145/3491101.3519665

[44] Stefan Van Der Walt, S Chris Colbert, and Gael Varoquaux. 2011. The NumPy array: a structure for efficient numerical computation. *Computing in science & engineering* 13, 2 (2011), 22–30.
[45] Guido Van Rossum et al. 2007. Python Programming Language.. In *USENIX annual technical conference*, Vol. 41. Santa Clara, CA, 1–36.
[46] J. S. Vetter, R. Brightwell, M. Gokhale, P. McCormick, R. Ross, J. Shalf, K. Antypas, D. Donofrio, T. Humble, C. Schuman, B. Van Essen, S. Yoo, A. Aiken, D. Bernholdt, S. Byna, K. Cameron, F. Cappello, B. Chapman, A. Chien, M. Hall, R. Hartman-Baker, Z. Lan, M. Lang, J. Leidel, S. Li, R. Lucas, J. Mellor-Crummey, P. Peltz Jr., T. Peterka, M. Strout, and J. Wilke. 2018. *Extreme Heterogeneity 2018 - Productive Computational Science in the Era of Extreme Heterogeneity: Report for DOE ASCR Workshop on Extreme Heterogeneity.* Technical Report. USDOE Office of Science (SC) (United States). https://doi.org/10.2172/1473756
[47] Yaqing Wang, Quanming Yao, James T. Kwok, and Lionel M. Ni. 2020. Generalizing from a Few Examples: A Survey on Few-Shot Learning. *ACM Comput. Surv.* 53, 3, Article 63 (jun 2020), 34 pages. https://doi.org/10.1145/3386252
[48] Michel Wermelinger. 2023. Using GitHub Copilot to Solve Simple Programming Problems. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (Toronto ON, Canada) *(SIGCSE 2023).* Association for Computing Machinery, New York, NY, USA, 172–178. https://doi.org/10.1145/3545945.3569830
[49] Burak Yetistiren, Isik Ozsoy, and Eray Tuzun. 2022. Assessing the Quality of GitHub Copilot's Code Generation. In *Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering* (Singapore, Singapore) *(PROMISE 2022).* Association for Computing Machinery, New York, NY, USA, 62–71. https://doi.org/10.1145/3558489.3559072

A ARTIFACT DESCRIPTION

The entire prompt input and resulting output sets are publicly available at: https://github.com/keitaTN/Copilot-hpc-kernels. Due to the rapid evolution and statistical nature of these technologies reproducibility and replicability of the present results is a challenging aspect that we expect to improve over time, but has no guarantees as of today. Hence, it's imperative that the reproducibility information is provided.

Some additional information:

- Experiments used the Visual Studio Code GitHub Copilot plugin service on three separate Linux systems using Ubuntu 22.04
- Experiments were carried between April 14th and April 21st of 2023.
- The raw dataset are identified by `<kernel>/<language>/<kernel>_outputs.<ext>` directory and file structure.
- Each Prompt used in this study is identified with the line comment `Prompt: <kernel> <programming model> <additional text>`.